

HASHDIT

HashDit

Smart Contract Security

Venus EBrake Spoke & bStock Pause Sentinel

Security Audit Report

Venus

23 Oct 2026 - 28 Oct 2026

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Period	23 Oct 2026 - 28 Oct 2026
Overall Risk	**Low**
Project Name	Venus EBrake Spoke & bStock Pause Sentinel
Github	https://github.com/VenusProtocol/venus-periphery/pull/81
Commit	Audit Commit: 95be65352fc90795f6f791c4757cc1bd43addc02 Fix Commit: 02cbc1261ad3d0e1bbbeb9be11fbf383db5cfd11

Smart Contract List

No.	Contract Name	Verdict	Details
1	BStockPauseSentinel.sol	Low	[L01]-Fixed [I02]-Ack
2	EBrake.sol	Informational	[I01]-Ack

Findings

[L01] BStockPauseSentinel Does Not Detect Compliance-Module Blocks on the vToken	
Contract	BStockPauseSentinel.sol
Severity	Low

Description	checkBStockPause() treats a bStock as non-transferable only when the token's PauseManager reports isTokenPaused(underlying):
	
	JavaScript
	`// BStockPauseSentinel.sol:192-198`
	`underlying = IVToken(market).underlying();`
	`if (underlying == address(0)) return (false, address(0), address(0));`
	
	`pauseManager = IBStockToken(underlying).pauseManager();`
	`if (pauseManager == address(0)) return (false, underlying, address(0));`
	
	`isPaused = IPauseManager(pauseManager).isTokenPaused(underlying);`
	
	The bStock implementation (SecuritiesToken) runs two independent checks on every transfer: the pause check and a compliance check on every party to the transfer:
	
	JavaScript
	`// SecuritiesToken._update`
	`function _update(address from, address to, uint256 value) internal override(ERC8056BaseUpgradeable) {`
	`  _checkTokenIsPaused();`
	`  if (from != msg.sender && to != msg.sender) {`
	`  _checkIsCompliant(msg.sender);`
	`  }`
	`  if (from != address(0)) {`
	`  _checkIsCompliant(from);`
	`  }`
	`  if (to != address(0)) {`
	`  _checkIsCompliant(to);`
	`  }`
	`  super._update(from, to, value);`
	`}`
	
	On BSC mainnet, all six bStocks listed on Venus (TSLAB, NVDAB, SPCXB, SKHYB in the Core pool; QQQB, SPYB in the Spoke pool) share the compliance module 0x53dBa7AaBDe774787A1F57236B235567dA8e14F4. It

keeps a global sanctions list (sanctionedAddresses / addToSanctionsList) and a per-token blocklist (blockedAddresses(token, account) / addToBlocklist). A redeem moves the bStock out of the vToken (from = vToken) and a mint moves it in (to = vToken), so sanctioning or blocklisting the vToken address blocks both directions exactly as a pause does. The sentinel only observes the pause check.

We simulated each state with an eth_call state override on vTSLAB (0x97421799419Eb782628e73e7220d8E0A207469a3):

- Current state (no override)
  - vTSLAB → user (redeem): OK
  - user → vTSLAB (mint): OK
  - PauseManager.isTokenPaused(TSLAB): false
- sanctionedAddresses[vTSLAB] = true
  - vTSLAB → user (redeem): reverts with UserSanctioned()
  - user → vTSLAB (mint): reverts with UserSanctioned()
  - PauseManager.isTokenPaused(TSLAB): false
- blockedAddresses[TSLAB][vTSLAB] = true
  - vTSLAB → user (redeem): reverts with UserBlocked()
  - user → vTSLAB (mint): reverts with UserBlocked()
  - PauseManager.isTokenPaused(TSLAB): false

Both lists can be updated by the compliance module's OPS_ROLE (0xA4E4975433038361eDc07D726022cb29A6aABC3e) and DEFAULT_ADMIN_ROLE (0x6f64F80B50efbf0f5f13D72d16eC17a59abBe5C6). Both are EOAs. The token's DEFAULT_ADMIN_ROLE (0x45e35Fe982F3869221b222Abea372fA97AA7679d, also an EOA) can additionally replace the compliance module or the PauseManager through setCompliance() / setPauseManager().

Impact

If the issuer side sanctions or blocklists a Venus vToken, the collateral becomes as unsellable as under a pause: redeems revert and liquidators cannot exit the seized vTokens. Possible causes include a compliance requirement, an operational mistake, or a compromised key. The sentinel does not react. Borrowers can keep borrowing at the full CF against that collateral until governance intervenes manually, and bad debt accrues if the price falls in the meantime. Because the module is shared, one transaction from a single EOA can affect all six bStock markets across the Core and Spoke pools at once. None of the vTokens is sanctioned or blocklisted today.

Recommendation

Also treat a market as paused when the token's compliance module blocks the vToken itself. The issuer can replace the module through setCompliance(), so wrap the calls in try/catch and fail closed if they revert:

```
JavaScript
isPaused =
IPauseManager(pauseManager).isTokenPaused(underlying);
if (!isPaused) {
    address compliance =
IBStockToken(underlying).compliance();
    try ICompliance(compliance).sanctionedAddresses(market)
returns (bool sanctioned) {
        isPaused = sanctioned;
    } catch {
        isPaused = true;
    }
    if (!isPaused) {
        try
ICompliance(compliance).blockedAddresses(underlying, market)
returns (bool blocked) {
            isPaused = blocked;
        } catch {
            isPaused = true;
        }
    }
}
```

If this case is meant to be handled manually (the Guardian calling EBrake.decreaseCF(address,uint256)), document that limitation. Also monitor the compliance module and the token's setCompliance / setPauseManager calls for any change that involves a Venus contract.

Status

Fixed in commit [02cbc12](). When the PauseManager does not report a pause, checkBStockPause() now also runs the token's own compliance check for the market, checkIsCompliant(underlying, market). Any revert from the compliance module counts as blocked, so a sanctioned or blocklisted vToken has its CF zeroed the same way as under a pause:

```
JavaScript
isPaused =
IPauseManager(pauseManager).isTokenPaused(underlying) ||
_isBlockedByCompliance(underlying, market);

function _isBlockedByCompliance(address underlying, address
market) private view returns (bool) {
```

```
    address compliance =
IBStockToken(underlying).compliance();
    try ICompliance(compliance).checkIsCompliant(underlying,
market) {
        return false;
    } catch {
        return true;
    }
}
```

[I01] Spoke Grants Implicitly Extend Every EBrake Permission Holder to Spoke Pools

Contract	EBrake.sol
Severity	Informational

Description

Before this change, every per-market EBrake function called the fixed COMPTROLLER, so a market of any other pool reverted. That revert was the only thing that kept each EBrake caller to the Core pool. EBrake now resolves the target from market.comptroller(). Its own ACM checks only ask whether the caller may call a function, not on which pool. Once a Spoke comptroller grants EBrake its four permissions, every existing EBrake permission holder can act on that Spoke as well, not only BStockPauseSentinel.

```javascript
// EBrake.sol:197 / :395
address comptroller = address(IVToken(market).comptroller());
...
IVToken(market).comptroller().setActionsPaused(markets,
actions, true);
DeviationSentinel already holds pauseBorrow, pauseSupply and
decreaseCF(address,uint256) on the BSC EBrake. Its config is
keyed by underlying token, and the market is supplied by the
keeper with no pool check:
// DeviationSentinel.sol:192-209
function handleDeviation(IVToken market) external onlyKeeper {
    address underlyingToken = market.underlying();
    DeviationConfig memory config =
tokenConfigs[underlyingToken];
    ...
    EBRAKE.pauseBorrow(address(market));   // or
decreaseCF(market, 0) + pauseSupply(market)
```

On BSC mainnet the Spoke pool "bStock index" (0x1e75332a3e2CC8e5ee9055aF4c737DA34Cb7cEDf) lists vUSDT, vUSDC and vU, and all three underlyings are configured and enabled in DeviationSentinel. Calling checkPriceDeviation() on these Spoke markets does not revert and returns the same prices as the corresponding Core markets, because both the configuration and the oracle lookup are keyed by token. With the previous EBrake, a handleDeviation() call on such a market reverts because the Core comptroller does not list it. With the new EBrake, the same call reaches the Spoke comptroller and pauses the market once the Spoke has granted EBrake its permissions. The project description states that DeviationSentinel "needs no review" because its code did not change, but its effective scope does.

Impact

The Spoke currently uses the same ResilientOracle as the Core pool, so the action taken on a Spoke market would follow the same price logic. The issue is that this scope expansion is implicit and was not decided explicitly. It also cannot be limited per caller with the current permission model, and

	it would become a real risk if a Spoke later adopts a different oracle or risk policy. The stablecoin markets in the current Spoke pool already have CF = 0, so in practice only pauseBorrow / pauseSupply would take effect there. No other pool is affected today: the BSC isolated pools and the non-Core pools on Ethereum, Arbitrum and Base have not granted EBrake permissions.
Recommendation	Decide which callers may act on non-Core pools and enforce it in EBrake, so that existing permission holders stay on the Core pool by default: JavaScript `function _resolveComptroller(address market) internal view returns (address comptroller) { comptroller = address(IVToken(market).comptroller()); if (comptroller != address(COMPTROLLER)) { _checkAccessAllowed("actOnOtherComptroller()"); } }` Use it in decreaseCF(address,uint256), _pauseSingleAction and _ensureSameComptroller, and grant the new permission only to BStockPauseSentinel and the Guardian. If DeviationSentinel is meant to cover Spoke pools, document that decision instead. Alternatively, deploy a separate EBrake instance per Spoke comptroller (COMPTROLLER = Spoke, IS_REMOTE_CHAIN = true, which needs no code change) and grant its permissions only to BStockPauseSentinel and the Guardian. Existing callers such as DeviationSentinel then remain bound to the Core EBrake by construction, at the cost of the sentinel routing each market to the matching EBrake and snapshots being split across instances.
Status	Acknowledged. The project accepts that EBrake permissions apply to every comptroller that grants EBrake access, so callers such as DeviationSentinel can also act on Spoke markets. The practical impact is limited today: the Spoke uses the same ResilientOracle as the Core pool, only trusted keepers can trigger DeviationSentinel, and EBrake can only tighten parameters.

[I02] Events Emitted Without a State Change	
Contract	BStockPauseSentinel.sol
Severity	Informational

Description	handleBStockPause() emits BStockPauseHandled on every call that finds the token paused, even when the CF is already zero and EBrake makes no change. setMarketMonitored() emits MarketMonitoringUpdated when adding a market that is already monitored or removing one that is not. setTrustedKeeper() emits TrustedKeeperUpdated when the status does not change.
	
	JavaScript
	`// BStockPauseSentinel.sol:114-115, 133-138, 162-164`
	`trustedKeepers[keeper] = isTrusted;`
	`emit TrustedKeeperUpdated(keeper, isTrusted);`
	`...`
	`if (monitored) {`
	`  monitoredMarkets.add(market);`
	`} else {`
	`  monitoredMarkets.remove(market);`
	`}`
	`emit MarketMonitoringUpdated(market, monitored);`
	`...`
	`EBRAKE.decreaseCF(market, 0);`
	`emit BStockPauseHandled(market, underlying, pauseManager);`
	
	Impact
	Off-chain monitoring that relies on these events can mistake repeated calls for new state changes. There is no on-chain impact.

Recommendation	Emit only on an actual change, and document that monitoring should key on EBrake's CollateralFactorDecreased for CF changes:
	
	JavaScript
	`bool changed = monitored ? monitoredMarkets.add(market) :`
	`monitoredMarkets.remove(market);`
	`if (changed) emit MarketMonitoringUpdated(market, monitored);`
	
	`if (trustedKeepers[keeper] == isTrusted) return;`

Status	Acknowledged. The project accepts that the sentinel emits events without a state change. The repeat-call behaviour of BStockPauseHandled is documented in its NatSpec, and off-chain monitoring can rely on EBrake's CollateralFactorDecreased for actual CF changes.